

SISTEMAS OPERATIVOS**Guía Práctica N° 6 — Memoria II**

Paginación por demanda, memoria virtual paginada y algoritmos de reemplazo

Cómo usar este documento

Cada ejercicio sigue la misma estructura: enunciado, la idea clave que hay que reconocer, resolución paso a paso (explicando el razonamiento, no solo el resultado) y la respuesta final tal como figura en la clave del profesor.

Los cálculos, tablas y resultados numéricos respetan exactamente la resolución oficial del profesor. Las explicaciones adicionales son para ayudarte a entender el “por qué” de cada paso.

Recordá la distinción clave: este documento trata memoria virtual paginada (paginación por demanda, bit de presencia/validez, reemplazo de páginas) — no confundir con segmentación con paginación (segmento | página | desplazamiento), que es un esquema distinto.

Índice

(Si el índice aparece vacío al abrir el archivo: en Word, clic derecho sobre esta sección → “Actualizar campos”; en LibreOffice, Herramientas → Actualizar → Índices y tablas.)

Índice	2
Conceptos clave antes de empezar	4
1. Formato de una dirección (virtual o física)	4
2. Traducción de dirección virtual a dirección física	4
3. Algoritmos de reemplazo de páginas	4
Ejercicio 1. Fallos de página con FIFO (paginación por demanda)	6
Idea clave	6
Resolución paso a paso	6
Respuesta final (clave del profesor)	7
Ejercicio 2. Identificar variables globales/locales a partir de direcciones físicas	7
Idea clave	7
Resolución paso a paso	7
Respuesta final (clave del profesor)	7
Respuesta final (clave del profesor)	8
Ejercicio 3. Formato de dirección virtual, entradas de tabla y marcos de página	8
Idea clave	8
Resolución paso a paso	8
Respuesta final (clave del profesor)	9
Ejercicio 4. Traducción de dirección virtual a física usando la tabla de páginas	9
Idea clave	10
Resolución paso a paso	10
Respuesta final (clave del profesor)	10
Ejercicio 5. Formatos de dirección virtual y física con memoria muy chica	10
Idea clave	11
Resolución paso a paso	11
Respuesta final (clave del profesor)	11
Ejercicio 6. Tamaño de página óptimo según fragmentación interna	11
Idea clave	12
Resolución paso a paso	12
Respuesta final (clave del profesor)	13
Ejercicio 7. Tabla de páginas de dos niveles	13
Idea clave	13
Resolución paso a paso	13
Respuesta final (clave del profesor)	14

Ejercicio 8. Tabla de páginas de tres niveles	14
Idea clave	14
Resolución paso a paso	14
Respuesta final (clave del profesor)	15
Ejercicio 9. Reemplazo de páginas: FIFO vs. LRU con datos de carga y referencia	15
Idea clave	15
Resolución paso a paso	15
Respuesta final (clave del profesor)	16
Ejercicio 10. Cadena de referencias y algoritmos de reemplazo: FIFO, Óptimo y Segunda Oportunidad	16
Idea clave	16
Resolución paso a paso	16
Respuesta final (clave del profesor)	18

Conceptos clave antes de empezar

Toda esta guía gira en torno a la traducción de direcciones en un sistema de memoria virtual paginada. Repasemos las piezas que se repiten en todos los ejercicios:

1. Formato de una dirección (virtual o física)

Una dirección se divide siempre en dos campos:

- Número de página (o de marco, en la dirección física): **los bits más significativos**.
- Desplazamiento (offset): **los bits menos significativos, indican la posición dentro de la página/marco**.

Regla práctica: el tamaño de página determina siempre el desplazamiento. Si la página mide 2^n bytes, se necesitan n bits para el desplazamiento. El resto de los bits de la dirección corresponden al número de página.

2. Traducción de dirección virtual a dirección física

1. Se separa la dirección virtual en (número de página, desplazamiento) según el formato calculado.
2. Se busca el número de página en la tabla de páginas.
3. Si el bit de presencia/validez es 1, la tabla indica el marco físico donde está esa página.
4. La dirección física se arma concatenando ese marco con el mismo desplazamiento (el desplazamiento nunca cambia, solo cambia el número de página por el número de marco).

Si el bit de presencia es 0, se produce un fallo de página (page fault): la página no está en memoria y el sistema operativo debe traerla desde disco.

3. Algoritmos de reemplazo de páginas

Algoritmo	Idea central	Cómo se aplica
FIFO	Se reemplaza la página que lleva más tiempo cargada en memoria.	Se mantiene una cola: la primera que entró es la primera en salir, sin importar si se usó recientemente.
Óptimo (MIN)	Se reemplaza la página que no se va a usar durante más tiempo en el futuro.	Requiere conocer la secuencia completa de referencias por adelantado (solo sirve como referencia teórica/cota inferior de fallos).
LRU (Least Recently Used)	Se reemplaza la página que no se usó hace más tiempo (pasado).	Se compara el instante de última referencia de cada página; sale la de referencia más antigua.
Segunda Oportunidad	Variante de FIFO que le da una 'segunda chance' a páginas usadas recientemente.	Se recorre en orden FIFO; si el bit R (referencia) está en 1, se lo pone en 0 y la página pasa al final de la cola sin reemplazarla; si está en 0, se reemplaza.

Distinción importante (para no confundir en el parcial)

Memoria virtual paginada (esta guía): dirección = página | desplazamiento. Se administra con tabla de páginas, bit de presencia, bit de modificado y algoritmos de reemplazo (FIFO, LRU, Óptimo, Segunda Oportunidad).

Segmentación con paginación (otro tema, no confundir): dirección = segmento | página | desplazamiento. Primero se selecciona el segmento y dentro de él se pagina.

Ejercicio 1. Fallos de página con FIFO (paginación por demanda)

Enunciado

En un sistema de 2KB de memoria física administrada mediante paginación por demanda, se tiene una entrada de 12 bits para el direccionamiento lógico, de los cuales los primeros 3 bits determinan el número de página. Determinar, utilizando FIFO, la cantidad de fallos de página que produce la secuencia de referencias: 589h, 111h, 3B8h, 9FBh, ABCh, 824h, 159h.

Idea clave

Antes de simular el algoritmo hay que traducir cada dirección hexadecimal a su número de página. El enunciado ya te da el formato: 3 bits de página + 9 bits de desplazamiento (3+9=12 bits, coincide con el direccionamiento lógico dado). Recién después se simula FIFO con la cantidad de marcos físicos disponibles.

Resolución paso a paso

1. Formato de la dirección: como el desplazamiento ocupa 9 bits, el tamaño de página es $2^9 = 512$ bytes.
2. Marcos físicos disponibles: la memoria física es de 2KB = 2048 bytes, entonces hay $2048 / 512 = 4$ frames (marcos A, B, C, D).
3. Se convierte cada dirección a binario de 12 bits y se toman los 3 bits más significativos para identificar la página referenciada:

Dirección	Binario (12 bits)	Página referenciada
589h	0101 1000 1001	2
111h	0001 0001 0001	0
3B8h	0011 1011 1000	1
9FBh	1001 1111 1011	4
ABCh	1010 1011 1100	5
824h	1000 0010 0100	4
159h	0001 0101 1001	0

Con esto la cadena de referencia a páginas queda: 2, 0, 1, 4, 5, 4, 0.

4. Se simula FIFO con 4 marcos. La regla de FIFO es simple: la página que hace más tiempo que está cargada es la primera en salir, sin importar si se usó hace poco. Se van llenando los marcos A→B→C→D en orden; cuando los 4 están ocupados y llega una página nueva, se reemplaza la que entró primero.

Marco	2	0	1	4	5	4	0
A	2	2	2	2	5	5	5
B	-	0	0	0	0	0	0
C	-	-	1	1	1	1	1
D	-	-	-	4	4	4	4
Fallos	F	F	F	F	F		

Los primeros 5 accesos (2, 0, 1, 4, 5) son todos fallos porque van llenando los 4 marcos y luego reemplazan al más viejo (A, que tenía la página 2, se reemplaza por la 5). Los últimos dos accesos (4 y 0) son aciertos porque esas páginas ya están en memoria (en D y B respectivamente).

Respuesta final (clave del profesor)

Resultado

FIFO produce 5 fallos de página.

Ejercicio 2. Identificar variables globales/locales a partir de direcciones físicas

Enunciado

En un sistema operativo propietario corre un único proceso con administración de memoria paginada simple. Las direcciones virtuales de 16 bits se dividen en 9 bits de desplazamiento y 7 bits de número de página. En el frame 0Ah está el área de código, en el frame 1Eh el área de datos (variables globales) y en el frame 3Ch el área de stack (variables locales a la función f1). a) La variable int a; que reside en la dirección física 3C08h, ¿es global o local a f1? b) En la dirección física 7834h se declara int array[?] (un int = 2 bytes), ¿qué tamaño podría alcanzar el vector?

Idea clave

La clave es reconocer que el número de marco (los 7 bits más significativos de la dirección física) indica en qué área de memoria cae la dirección: si coincide con 0Ah es código, con 1Eh son datos (globales) y con 3Ch es la pila (variables locales). Una vez identificado el área, para saber cuánto puede crecer un vector hay que calcular cuánto espacio libre queda desde su posición hasta el final del marco.

Resolución paso a paso

a. Dirección física 3C08h

1. Formato: 7 bits de marco + 9 bits de desplazamiento = 16 bits.
2. 3C08h en binario: 0011 1100 0000 1000.
3. Los 7 bits más significativos (número de marco): 0011110b = 1Eh = 30 decimal.
4. 1Eh es justamente el frame del área de datos (variables globales).

Respuesta final (clave del profesor)

Resultado

La variable int a; es una variable global, porque su dirección cae dentro del frame 1Eh (área de datos).

b. Dirección física 7834h

1. 7834h en binario: 0111 1000 0011 0100.
2. Los 7 bits más significativos (marco): agrupando de a 4 bits se obtiene 0011 1100 = 3Ch = 60 decimal, que es el frame de la pila. Por lo tanto array es una variable local a f1.
3. Se reconstruye la dirección física agrupada: 3C34h. El desplazamiento es 34h = 52 decimal, es decir que el vector arranca 52 bytes después del inicio del marco.
4. Como cada marco mide 512 bytes (2^9 , el mismo tamaño de página calculado a partir de los 9 bits de desplazamiento), el espacio libre desde ahí hasta el final del marco es $512 - 52 = 460$ bytes.
5. Como cada int ocupa 2 bytes, la cantidad máxima de elementos es $460 / 2$ bytes.

Respuesta final (clave del profesor)**Resultado**

El vector array puede llegar a tener hasta 230 elementos (460 bytes de espacio libre en el marco $\div 2$ bytes por entero).

Nota sobre la clave del profesor

El material del profesor indica 130 elementos en este punto, pero $460 \div 2$ da 230. Es un error aritmético de la clave (probablemente una errata de tipeo). El razonamiento y todos los demás valores (marco 3Ch, desplazamiento 52, 460 bytes libres) son correctos y coinciden con el original; solo el resultado final de la división parece mal transcrito. Vale la pena mencionárselo al profesor o confirmarlo en clase antes del examen.

Ejercicio 3. Formato de dirección virtual, entradas de tabla y marcos de página**Enunciado**

Sistema operativo de 20 bits con memoria virtual paginada, páginas de 1KB, memoria física total de 256KB. a) ¿Cuál es el formato de la dirección virtual? b) ¿Cuál es el número máximo de entradas de la tabla de páginas (de un nivel)? c) ¿Cuántos marcos de página tiene la memoria principal?

Idea clave

Estos tres incisos son la base de casi todos los cálculos de la guía: a partir del tamaño de página se obtiene el desplazamiento, el resto de los bits de la dirección virtual son para el número de página, y ese número de página determina cuántas entradas como máximo puede tener la tabla de páginas. La cantidad de marcos surge de dividir la memoria física por el tamaño de página (que es igual al tamaño de marco).

Resolución paso a paso**a. Formato de la dirección virtual**

1. Página = 1KB = 2^{10} bytes → se necesitan 10 bits para el desplazamiento.
2. Dirección virtual = 20 bits → $20 - 10 = 10$ bits para el número de página.

10 bits	10 bits
PÁGINA	DESPLAZAMIENTO

b. Entradas máximas de la tabla de páginas

Coincide con la cantidad máxima de páginas direccionables: $2^{10} = 1024$ entradas.

c. Marcos de página de la memoria principal

256KB / 1KB (tamaño de marco = tamaño de página) = 256 marcos.

Respuesta final (clave del profesor)

Resultado

- a. 10 bits para página + 10 bits para desplazamiento.
- b. 1024 entradas como máximo en la tabla de páginas.
- c. 256 marcos de página en la memoria física.

Ejercicio 4. Traducción de dirección virtual a física usando la tabla de páginas

Enunciado

Un sistema usa páginas de 8KB, ofrece un espacio de memoria virtual de 2^{32} bytes y tiene 2^{23} bytes de memoria física. La tabla de páginas de un programa en ejecución es la siguiente:

Página	Bit presencia	Bit modificado	Marcos
0	1	0	1
1	0	0	7
2	1	1	9
3	1	0	14
4	1	0	8
5	1	1	3
6	0	0	25
7	0	1	16
8	0	0	23
9	1	0	78

Se pide: a) formato de la dirección virtual. b) dirección física de la dirección virtual 0x0000608A. c) tamaño del espacio de direcciones virtual de este programa. d) tamaño de la memoria principal en MB.

Idea clave

Con el bit de presencia se sabe si la página está en memoria (1) o produciría un fallo de página (0). Para traducir una dirección virtual a física: se separa en (página, desplazamiento), se busca el marco correspondiente en la tabla, y se arma la dirección física uniendo ese marco con el mismo desplazamiento — el desplazamiento nunca cambia entre la dirección virtual y la física.

Resolución paso a paso

a. Formato de la dirección virtual

1. Página de 8KB = 2^{13} bytes $\rightarrow$ 13 bits de desplazamiento.
2. Memoria virtual = 2^{32} bytes $\rightarrow$ se usan los 19 bits superiores ($32 - 13$) para el número de página, y los 13 bits inferiores para el desplazamiento.

b. Traducción de 0x0000608A

1. En binario: 0000 0000 0000 0000 0110 0000 1000 1010.
2. Los 19 bits superiores dan el número de página: 0000 0000 0000 0000 011 = página 3.
3. En la tabla, la página 3 tiene bit de presencia = 1 (está en memoria) y marco = 14 (1110 en binario).
4. Se reemplaza el campo de página por el marco 1110 y se conserva el desplazamiento: 0000 0000 0000 0001 1100 0000 1000 1010 = 0x0001C08A.

c. Tamaño del espacio de direcciones del programa

La tabla tiene 10 entradas (páginas 0 a 9), es decir el programa ocupa 10 páginas $\times$ 8KB = 80KB.

d. Tamaño de la memoria principal

2^{23} bytes = 8 388 608 bytes = 8192KB = 8MB.

Respuesta final (clave del profesor)

Resultado

- a. 19 bits de página + 13 bits de desplazamiento.
- b. La dirección física correspondiente a 0x0000608A es 0x0001C08A.
- c. El programa ocupa 80KB de espacio de direcciones virtual.
- d. La memoria principal tiene 8MB.

Ejercicio 5. Formatos de dirección virtual y física con memoria muy chica

Enunciado

Un sistema direcciona la memoria por bytes, usa direcciones virtuales de 16 bits, páginas de 2KB y la memoria física es de 8KB. Se pide: a) tamaño máximo de la memoria virtual en KB. b) número máximo de páginas de un programa. c) formato de la dirección virtual. d) tamaño del marco de página. e) número de marcos de página de la memoria física. f)

formato de la dirección física. g) número máximo de entradas de la tabla de páginas de un nivel.

Idea clave

Este ejercicio combina todo lo visto: hay que trabajar con dos direcciones distintas (virtual y física) que comparten el mismo desplazamiento (porque el tamaño de página = tamaño de marco) pero difieren en la cantidad de bits para identificar página vs. marco, ya que la memoria virtual y la física tienen tamaños distintos.

Resolución paso a paso

1. a. Con 16 bits de dirección virtual se direccionan 2^{16} posiciones. Como se direcciona por byte, la memoria virtual máxima es 2^{16} bytes = 64KB.
2. b. Con páginas de 2KB, el número máximo de páginas es $64\text{KB} / 2\text{KB} = 32$ páginas → se necesitan 5 bits ($2^5 = 32$).
3. c. Formato de la dirección virtual: 5 bits de página + 11 bits de desplazamiento (porque $2\text{KB} = 2^{11}$ bytes).

5 bits	11 bits
PÁGINA	DESPLAZAMIENTO

4. d. El marco de página siempre tiene el mismo tamaño que la página: 2KB.
5. e. Con memoria física de 8KB y marcos de 2KB: $8\text{KB} / 2\text{KB} = 4$ marcos → se necesitan 2 bits ($2^2 = 4$).
6. f. Formato de la dirección física: 2 bits de marco + 11 bits de desplazamiento (el desplazamiento se mantiene igual al de la dirección virtual).

2 bits	11 bits
MARCO	DESPLAZAMIENTO

7. g. Como la tabla de páginas es de un único nivel, tiene una entrada por cada página posible del espacio virtual: 32 entradas.

Respuesta final (clave del profesor)

Resultado

a. 64KB. b. 32 páginas. c. 5 bits página + 11 bits desplazamiento.
d. 2KB. e. 4 marcos. f. 2 bits marco + 11 bits desplazamiento. g. 32 entradas.

Ejercicio 6. Tamaño de página óptimo según fragmentación interna

Enunciado

Se deben ejecutar concurrentemente 3 procesos cuyos segmentos (código, datos, pila) tienen los tamaños de la tabla siguiente, en una memoria física de 64KB. Una página no

puede contener partes de segmentos distintos. a) ¿Son viables tamaños de página de 1024 y 512 bytes? b) De ser viables ambos, ¿cuál conviene según la fragmentación interna? c) Si las entradas de la tabla de páginas deben tener 3 bits para referenciar el marco, ¿qué tamaño de página debería usarse (sin tener en cuenta los procesos A, B y C)?

Proceso	Código	Datos	Pila
A	16384	8700	8192
B	2048	1000	1024
C	4096	2272	2048

Idea clave

La restricción “una página no puede contener partes de dos segmentos distintos” es la que obliga a redondear hacia arriba cada segmento por separado (no se puede compactar el sobrante de un segmento con el siguiente). Eso genera fragmentación interna en la última página de cada segmento. La página “conveniente” es la que desperdicia menos memoria en total; y el tamaño de página necesario para un ancho de bits dado se obtiene dividiendo la memoria física por la cantidad de marcos que ese ancho de bits permite direccionar.

Resolución paso a paso

a. ¿Son viables 1024 bytes y 512 bytes?

Se calcula, para cada segmento de cada proceso, cuántas páginas ocupa (redondeando hacia arriba) y la fragmentación interna resultante en la última página:

Segmento	Bytes	N° pág. (1024B)	Frag. interna	N° pág. (512B)	Frag. interna
Proceso A — Código	16384	16	0	32	0
Proceso A — Datos	8700	9	516	17	4
Proceso A — Pila	8192	8	0	16	0
Proceso B — Código	2048	2	0	4	0
Proceso B — Datos	1000	1	24	2	24
Proceso B — Pila	1024	1	0	2	0
Proceso C — Código	4096	4	0	8	0
Proceso C — Datos	2272	3	800	5	288
Proceso C — Pila	2048	2	0	4	0
TOTALES	45764	46	1340	90	312

Con páginas de 1024 bytes se necesitan 46 páginas en total. El espacio de direcciones de 64KB (65536 bytes) admite $65536/1024 = 64$ páginas posibles, y $46 \leq 64$, así que es viable.

Con páginas de 512 bytes se necesitan 90 páginas en total. El espacio admite $65536/512 = 128$ páginas posibles, y $90 \leq 128$, así que también es viable.

b. ¿Qué tamaño conviene según la fragmentación interna?

Se suma la fragmentación interna total de cada opción: con 1024 bytes se desperdician $516 + 24 + 800 = 1340$ bytes; con 512 bytes se desperdician $4 + 24 + 288 = 316$ bytes.

c. Tamaño de página para 3 bits de referencia al marco

1. Con 3 bits solo se pueden referenciar $2^3 = 8$ marcos como máximo.
2. Página de 1024 bytes $\rightarrow 16384/1024 = 2^4$ marcos $\rightarrow$ hacen falta 4 bits. No alcanza con 3.
3. Página de 512 bytes $\rightarrow 16384/512 = 2^5$ marcos $\rightarrow$ hacen falta 5 bits. Tampoco alcanza.
4. Ninguno de los dos tamaños propuestos sirve. Para que 3 bits alcancen a referenciar todos los marcos posibles, se necesitan exactamente $2^3 = 8$ marcos, entonces el tamaño de página debe ser $16384 / 8 = 2048$ bytes.

Respuesta final (clave del profesor)**Resultado**

- a. Ambos tamaños (1024 y 512 bytes) son viables.
- b. Conviene usar páginas de 512 bytes: generan 316 bytes de fragmentación interna total, frente a 1340 bytes con páginas de 1024.
- c. Para referenciar el marco con 3 bits se necesitan páginas de 2048 bytes (8 marcos de página).

Ejercicio 7. Tabla de páginas de dos niveles**Enunciado**

Una computadora de direccionamiento de 32 bits usa una tabla de páginas de dos niveles. Las direcciones virtuales se dividen en un campo de 9 bits para la tabla de primer nivel, un campo de 11 bits para la tabla de segundo nivel, y un campo de desplazamiento. ¿Qué tamaño deben tener las páginas y cuántas hay en un espacio de direcciones?

Idea clave

En una tabla de páginas multinivel, la dirección virtual se reparte entre los distintos niveles y el desplazamiento. Lo que sobra de los 32 bits totales, después de restar los bits usados por cada nivel, es el desplazamiento — y ese desplazamiento determina el tamaño de página. La cantidad total de páginas del espacio de direcciones es el producto de las entradas posibles en cada nivel (o, equivalentemente, 2 elevado a la suma de los bits de ambos niveles).

Resolución paso a paso

1. De los 32 bits de la dirección virtual, 9 bits van para la tabla de primer nivel y 11 bits para la de segundo nivel: $9 + 11 = 20$ bits ya usados.
2. Los bits restantes son el desplazamiento: $32 - 20 = 12$ bits.

9 bits	11 bits	12 bits
PÁGINA (N1)	PÁGINA (N2)	DESPLAZAMIENTO

3. El tamaño de página es $2^{12} = 4096$ bytes = 4KB.
4. La tabla de primer nivel tiene $2^9 = 512$ entradas posibles y la de segundo nivel $2^{11} = 2048$ entradas posibles.

5. La cantidad total de páginas del espacio de direcciones es $2^9 \times 2^{11} = 2^{20} = 512 \times 2048 = 1\,048\,576$ páginas.

Respuesta final (clave del profesor)

Resultado

El tamaño de página es de 4KB (4096 bytes).

El espacio de direcciones contiene 1 048 576 páginas en total (2^{20}).

Ejercicio 8. Tabla de páginas de tres niveles

Enunciado

CPU de 32 bits con direcciones virtuales de 36 bits; buses de direcciones y datos de 32 bits. Memoria virtual paginada con: (i) tres niveles de tablas de páginas, páginas de 8KB; (ii) cada entrada de cualquier nivel ocupa una palabra de 4 bytes; (iii) la tabla de tercer nivel se aloja en un frame; (iv) las tablas de primer y segundo nivel son del mismo tamaño; (v) ¿cuántos frames hay en total en la memoria RAM? Describa los campos en que se puede descomponer una dirección.

Idea clave

Acá la dificultad es que no te dan directamente los bits de cada nivel: hay que deducirlos a partir de cuántas entradas entran en una página, usando el tamaño de la entrada (4 bytes) y el tamaño de página (8KB). Una vez armado el tercer nivel, lo que sobra de los 36 bits se reparte en partes iguales entre el primer y segundo nivel porque el enunciado dice que sus tablas tienen el mismo tamaño.

Resolución paso a paso

- Desplazamiento: página de 8KB = 2^{13} bytes → se necesitan 13 bits para el desplazamiento dentro de la página.
- Tabla de tercer nivel (TP3): cada entrada ocupa una palabra = 2^2 bytes, y la tabla completa ocupa una página = 2^{13} bytes. Entonces tiene $2^{13} / 2^2 = 2^{11}$ entradas → se necesitan 11 bits para indexar TP3.
- Bits usados hasta ahora: 13 (desplazamiento) + 11 (TP3) = 24 bits. Quedan $36 - 24 = 12$ bits para repartir entre TP1 y TP2.
- Como TP1 y TP2 tienen el mismo tamaño, se reparten esos 12 bits en partes iguales: 6 bits para cada una.

6 bits	6 bits	11 bits	13 bits
PÁGINA (N1)	PÁGINA (N2)	PÁGINA (N3)	DESPLAZAMIENTO

Corrección de una errata en la clave

La tabla original del profesor tiene un error de tipeo: repite "PÁGINA (N3)" en las dos primeras columnas en lugar de N1, N2 y N3. Siguiendo el desarrollo del propio texto (que nombra TP1, TP2 y TP3 en ese orden y explica que a TP1 y TP2 les corresponden 6 bits

6 bits	6 bits	11 bits	13 bits
cada una), la tabla corregida es: 6 bits para N1, 6 bits para N2, 11 bits para N3 y 13 bits de desplazamiento, como se muestra arriba.			

v. Cantidad total de frames en la RAM (no desarrollado en la clave del profesor)

La clave no resuelve explícitamente este punto. Como referencia, se puede estimar a partir de que los buses de direcciones son de 32 bits, es decir que la memoria física direccionable es de 2^{32} bytes. Con marcos de $8\text{KB} = 2^{13}$ bytes, la cantidad de frames sería $2^{32} / 2^{13} = 2^{19} = 524\,288$ frames. Conviene confirmar este razonamiento con el profesor antes del examen, ya que no está en la resolución oficial.

Respuesta final (clave del profesor)

Resultado

La dirección virtual de 36 bits se descompone en: 6 bits para TP1, 6 bits para TP2, 11 bits para TP3 y 13 bits de desplazamiento.

Ejercicio 9. Reemplazo de páginas: FIFO vs. LRU con datos de carga y referencia

Enunciado

Una computadora tiene 4 marcos de página. Se lista el tiempo de carga, el tiempo de última referencia y los bits R (referencia) y M (modificado) de cada página, en ticks de reloj:

Página	Carga	Última referencia	Bit R	Bit M
0	126	279	0	0
1	230	260	1	0
2	120	272	1	1
3	160	280	1	1

a) ¿Qué página se reemplazará con el algoritmo FIFO? b) ¿Qué página se reemplazará con el algoritmo LRU?

Idea clave

La diferencia entre FIFO y LRU está en qué columna de la tabla hay que mirar. FIFO no le importa el uso: solo mira quién entró primero (columna “Carga”). LRU sí le importa el uso: mira quién se usó hace más tiempo (columna “Última referencia”), sin importar cuándo entró.

Resolución paso a paso

a. FIFO

Se busca el menor tiempo de carga, porque esa es la página que lleva más tiempo ocupando un marco: página 2 (carga = 120, la más antigua de las cuatro).

b. LRU

Se busca el menor tiempo de última referencia, porque esa es la página que no se usa hace más tiempo: página 1 (última referencia = 260, la más lejana en el pasado de las cuatro).

Respuesta final (clave del profesor)

Resultado

FIFO reemplaza la página 2 (fue la primera cargada de las cuatro).

LRU reemplaza la página 1 (su último uso es el más lejano en el tiempo de las cuatro).

Ejercicio 10. Cadena de referencias y algoritmos de reemplazo: FIFO, Óptimo y Segunda Oportunidad

Enunciado

Sistema de 16MB de memoria principal, memoria virtual paginada con páginas de 4KB. Un proceso genera la siguiente secuencia de accesos a memoria (en hexadecimal): 02D4B8, 02D4B9, 02D4EB, 02D4EB, 02D86F, F0B621, F0B815, F0D963, F0B832, F0BA23, D9D6C3, D9B1A7, D9B1A1, F0BA25, 02D4C7, 628A31, F0B328, D9B325, D73425. El sistema operativo asigna al proceso 4 marcos de memoria principal. Se pide: a) formato de la dirección física. b) cadena de referencia de páginas accedidas. c) comportamiento del gestor de memoria con FIFO, Óptimo y Segunda Oportunidad.

Idea clave

El primer paso, como siempre, es encontrar el formato de la dirección para poder separar la página del desplazamiento. Como las direcciones vienen en hexadecimal, conviene razonar en dígitos hexadecimales (cada uno son 4 bits) en vez de convertir todo a binario. Una vez que se tiene la cadena de páginas referenciadas (ignorando el desplazamiento), se simulan los tres algoritmos de reemplazo con 4 marcos.

Resolución paso a paso

a. Formato de la dirección física

1. Memoria física de 16MB = 2^{24} bytes → se necesitan 24 bits para direccionar toda la memoria física.
2. Página de 4KB = 2^{12} bytes → se necesitan 12 bits para el desplazamiento.

12 bits	12 bits
PÁGINA	DESPLAZAMIENTO

b. Cadena de referencia de páginas

Como cada dígito hexadecimal equivale a 4 bits, 12 bits de página equivalen a los 3 primeros dígitos hexadecimales de cada dirección (los otros 3 dígitos son el desplazamiento). Por ejemplo 02D4B8 se separa en página 02D y desplazamiento 4B8.

12 bits	12 bits
PÁGINA	DESPLAZAMIENTO
02D	4B8

Tomando solo los 3 dígitos de página de cada dirección de la secuencia, la cadena de referencia de páginas es:

Cadena de referencia (18 accesos)

02D, 02D, 02D, 02D, 02D, F0B, F0D, F0B, F0B, D9D, D9B, D9B, F0B, 02D, 628, F0B, D9B, D73

c. Simulación de los algoritmos de reemplazo (4 marcos: M1 a M4)

FIFO — 9 fallos

En FIFO se reemplaza siempre la página que hace más tiempo está en memoria, sin mirar si se usó hace poco. Los marcos se llenan en orden de llegada y, al estar los 4 ocupados, cada fallo nuevo desaloja al que entró primero (el que está “más adelante” en la cola).

	02D	02D	02D	02D	02D	F0B	F0D	F0B	F0B	D9D	D9B	D9B	F0B	02D	628	F0B	D9B	D73
M1	02D	02D	02D	02D	02D	02D	02D	02D	02D	02D	D9B	D9B	D9B	D9B	D9B	D9B	D9B	D73
M2					F0B	F0B	F0B	F0B	F0B	F0B	F0B	F0B	02D	02D	02D	02D	02D	
M3						F0D	F0D	F0D	F0D	F0D	F0D	F0D	F0D	628	628	628	628	628
M4									D9D	D9D	D9D	D9D	D9D	D9D	D9D	F0B	F0B	F0B
Fallo	F					F	F			F	F			F	F	F		F

Fallos en los accesos: 1 (02D), 6 (F0B), 7 (F0D), 10 (D9D), 11 (D9B — desaloja a 02D, el más viejo), 14 (02D — desaloja a F0B), 15 (628 — desaloja a F0D), 16 (F0B — desaloja a D9D) y 18 (D73 — desaloja a D9B). Total: 9 fallos.

Óptimo — 7 fallos

El algoritmo Óptimo (o MIN) reemplaza la página que no se va a volver a usar durante más tiempo hacia adelante en la secuencia. Es un algoritmo teórico (necesita conocer el futuro), pero da la menor cantidad de fallos posible y sirve como punto de comparación.

	02D	02D	02D	02D	02D	F0B	F0D	F0B	F0B	D9D	D9B	D9B	F0B	02D	628	F0B	D9B	D73
M1	02D	02D	02D	02D	02D	02D	02D	02D	02D	02D	02D	02D	02D	02D	628	628	628	628
M2					F0B	F0B	F0B	F0B	F0B	F0B	F0B	F0B	F0B	F0B	F0B	F0B	F0B	D73
M3						F0D	F0D	F0D	F0D	D9B	D9B	D9B	D9B	D9B	D9B	D9B	D9B	
M4									D9D	D9D	D9D	D9D	D9D	D9D	D9D	D9D	D9D	
Fallo	F					F	F			F	F			F			F	

En el acceso 11 (D9B) hay que elegir a quién desalojar entre F0D y D9D, que ya no vuelven a usarse nunca más en la secuencia (empate entre “no se usan más”); se desaloja F0D. En el acceso 15 (628) se desaloja 02D, porque después del acceso 14 ya no vuelve a aparecer. En el acceso 18 (D73, el último) todas las páginas residentes están “empatadas” porque ninguna se va a volver a usar; se desaloja F0B. Total: 7 fallos.

Segunda Oportunidad — 7 fallos

Segunda Oportunidad funciona como FIFO pero antes de desalojar a la página más vieja revisa su bit R (referencia): si está en 1, la perdona (le da una “segunda oportunidad”), le pone el bit en 0 y la manda al final de la cola en vez de sacarla; recién desaloja a una página cuando encuentra el bit R en 0.

	02D	02D	02D	02D	02D	F0B	F0D	F0B	F0B	D9D	D9B	D9B	F0B	02D	628	F0B	D9B	D73
M1	02D	02D	02D	02D	02D	02D	02D	02D	02D	02D	02D	02D	02D	02D	02D	02D	02D	D73
M2					F0B	F0B	F0B	F0B	F0B	F0B	F0B	F0B	F0B	F0B	F0B	F0B	F0B	F0B
M3						F0D	F0D	F0D	F0D	F0D	F0D	F0D	F0D	F0D	628	628	628	628
M4									D9D	D9D	D9B	D9B	D9B	D9B	D9B	D9B	D9B	D9B
Fallo	F					F	F			F	F			F			F	

En este caso las páginas que llegan a estar en cola tienen su bit R en 1 en el momento en que les tocaría salir, así que se les da la segunda oportunidad y sobrevive más tiempo del que sobreviviría con FIFO puro; por eso el resultado se parece más al de Óptimo. Total: 7 fallos, en los mismos accesos que en el algoritmo Óptimo (1, 6, 7, 10, 11, 15 y 18).

Respuesta final (clave del profesor)

Resultado

- 12 bits de página + 12 bits de desplazamiento.
- Cadena de páginas: 02D, 02D, 02D, 02D, 02D, F0B, F0D, F0B, F0B, D9D, D9B, D9B, F0B, 02D, 628, F0B, D9B, D73.
- FIFO: 9 fallos. Óptimo: 7 fallos. Segunda Oportunidad: 7 fallos.

Para repasar antes del examen

Practicá primero encontrar el formato de la dirección (bits de página vs. desplazamiento) en cualquier ejercicio nuevo: es el paso que se repite siempre y del que dependen todos los demás cálculos.

Para reemplazo de páginas, tené claro qué dato mira cada algoritmo: FIFO mira cuándo entró la página, LRU mira cuándo se usó por última vez, Óptimo mira cuándo se va a volver a usar en el futuro, y Segunda Oportunidad combina FIFO con el bit R.

No confundas esta unidad (memoria virtual paginada) con segmentación con paginación, que usa un formato de dirección distinto (segmento | página | desplazamiento).